

# Chapter 10. Geo and Switchback Experiments

In [Part IV](#) of this book, you learned how to use repeated observations over time to aid in the process of causal inference. Now, in this chapter, you will approach the same problem from a different angle. What if, instead of having to use panel data to identify a treatment effect, you had to design an experiment to gather that data? [Part V](#) of this book is dedicated to alternative experimental design when simple A/B testing won't work.

For example, let's consider the marketing problem from the previous chapter. Remember that inferring the impact of marketing is challenging because you cannot randomize people who are not yet your customers. Online marketing provides you with attribution tools, but attribution is not the same as incrementality. In this case, a promising alternative is to conduct a geo-experiment: treat entire markets, such as a city or a state, while leaving others as control. This approach would provide you with panel data to which you could apply the techniques learned in [Part IV](#). However, in [Part IV](#), you took the panel data as given and did not learn how to best select the treated and control markets in such an experiment. In this chapter, you will cover that gap. The first part of this chapter will teach you how to select geographical

treatment units to get an effect estimate that approximates the effect you would have if the entire market (country, state) were treated.

The idea is to zoom out the unit of analysis from customers to cities or states. Of course, this comes at a price in terms of sample size since there are far more customers than there are cities. In some extreme cases, even randomizing cities may not be possible. For instance, if you are a small company operating locally, you may not have many markets in which to conduct an experiment. In the limit, you may have only a single unit of analysis. Fortunately, there is still a way out. Switchback experiments involve turning a treatment on and off multiple times. This approach can work even if you have only a single treatment unit. For example, if you are a small food delivery marketplace operating within a single city and you want to know what happens if you increase the delivery fee, you can increase and decrease prices multiple times and conduct a series of before-and-after analyses. The remainder of this chapter will expand on this idea and how to design a switchback experiment.

## Geo-Experiments

To motivate the use of geo-experiments, let's take the same data and example from the previous chapter. Again, you have the city as the unit and date as the time dimension; a

treated column, which marks if the city is eventually treated; and a post-treatment column, which marks the post-intervention period. You also have some auxiliary columns, like the population in that city (recorded in 2013, so fixed in time) and the state. Here, you'll work with the outcome: number of app downloads. And since the goal is to decide which city to treat, you'll discard the post-intervention period:

```
In [1]: import pandas as pd
        import numpy as np

        df = (pd.read_csv("./data/online_mkt.csv")
               .astype({"date": "datetime64[ns]"})
               .query("post==0"))

        df.head()
```

|   | app_download | population | city   |
|---|--------------|------------|--------|
| 0 | 3066.0       | 12396372   | sao_pa |
| 1 | 2701.0       | 12396372   | sao_pa |
| 2 | 1927.0       | 12396372   | sao_pa |
| 3 | 1451.0       | 12396372   | sao_pa |
| 4 | 1248.0       | 12396372   | sao_pa |

The objective is to select a group of cities that is representative of the total market. That way, if you treat that group, you'll get an idea of what would happen if the entire market (i.e., country) was treated. Before you do anything fancy, it's worth trying the simple things. If you have lots of geographical units (i.e., many cities), perhaps a simple A/B testing would work just fine. Simply choose at random a fraction of the cities to compose the treatment group. The only difference here is that you would be shifting the unit of analysis from people (potential customers) to entire cities.

Still, it's hard to know how many cities are enough. You could start with the sample size formula from [Chapter 2](#) (

$n = 16\sigma^2/\delta^2$ ). For instance, if you wish to detect an effect of 5%, it would tell you that you need about 40k cities to run that experiment:

```
In [2]: detectable_diff = df["app_download"].me
        sigma_2 = df.groupby("city")["app_downl

        np.ceil((sigma_2*16)/(detectable_diff))
```

```
Out[2]: 36663.0
```

But that formula doesn't take into account that each city has a different variance of the outcome (cities with more people have lower variance) nor that you can leverage repeated observations of the same unit to increase the precision of your estimate. Even so, 40k cities seems a lot more than the 50 cities that you have in this data. So what can you do if you are short of units?

When considering different experimental options, it's essential to take into account how you will interpret the results. For example, if you plan on using the difference-in-differences method, you can identify pairs of cities that have similar trends in the outcome variable and randomize within each pair. One city would receive the treatment, while the

other would serve as the control. However, it's worth noting that the diff-in-diff method estimates the average treatment effect on the treated (ATT). If you want to know the overall effect of the treatment, such as deploying the marketing campaign nationwide, the ATT estimate may not recover that. In this chapter, we will explore an idea that aims to maximize the external validity of the experiment by identifying a treatment group that is representative of the entire market.

## Synthetic Control Design

Finding a group of units whose time series can best approximate the outcome of interest is exactly what you've been doing with synthetic controls. So, it's not surprising that you can repurpose the method to find a synthetic treatment unit that approximates the average behavior of all units. To do so, you'll only need the data in matrix form, where the rows are the time periods and the columns are the cities,  $Y_{T,n}$  :

```
In [3]: df_piv = (df
          .pivot("date", "city", "app_c")
          .df_piv.head())
```

| city       | ananindeua | aparecida_de_goiania |
|------------|------------|----------------------|
| date       |            |                      |
| 2022-03-01 | 11.0       | 54.0                 |
| 2022-03-02 | 5.0        | 20.0                 |
| 2022-03-03 | 2.0        | 0.0                  |
| 2022-03-04 | 0.0        | 0.0                  |
| 2022-03-05 | 5.0        | 5.0                  |

Now, let's think of what you want to achieve. First, let's keep in mind that each city contributes differently for the average. To get the global average you first need to know how much each city contributes to it. This can be represented by a city weight vector  $\mathbf{f}$  where each entry  $i$  represents the size of city  $i$  as a share of the total market:

```
In [4]: f = (df.groupby("city")["population"].sum()
          / df.groupby("city")["population"].sum().sum())
```

Once  $\mathbf{f}$  is defined, you can see that your objective is first to find a set of weights  $\mathbf{w}$  such that:

$$\mathbf{Y}_{post}\mathbf{f} = \mathbf{Y}_{post}\mathbf{w}$$

Translating that to words, you want to find a combination of treatment cities that, when combined with weights  $\mathbf{w}$ , will give you the average outcome of the market. Leaving it like this, the obvious solution would be to set  $\mathbf{f} = \mathbf{w}$ , but that would mean treating every city, which would leave you without any control units to estimate the treatment effect, so you have to add a constraint that the number of nonzero elements in  $\mathbf{w}$  must be smaller than  $N$ , the number of cities:  $|\mathbf{w}|_0 < N$  ( $|\cdot|_0$  is the L0 norm, which is the number of nonzero elements). Moreover, the preceding objective is impossible to solve, since you don't observe the post-intervention values of  $\mathbf{Y}$ . Still, let's keep it as an ideal and see if you can somehow approximate it.

If the preceding objective finds the cities that you can use as treatment, your second objective is to find another group of cities, different from the first, that will also approximate the market average:

$$\begin{aligned} \mathbf{Y}_{post}\mathbf{f} &= \mathbf{Y}_{post}\mathbf{v} \\ \text{s.t. } w_i v_i &= 0 \quad \forall i, \end{aligned}$$



Combining both objectives, the nonzero elements in  $\mathbf{w}$  and  $\mathbf{v}$  would be the treatment and control cities, respectively. Also, this second objective introduces a constraint which states that you can't use the same city as a treatment and as a control. Once again, this objective is not feasible, as it looks at the post-treatment period.

Now, you just need to think a bit on how to swap those impossible objectives with one that you might be able to solve. The obvious first step is looking at the pre-intervention period. This means that you would find two disjoint groups of cities such that a weighted average of each approximates the market average. That alone would be enough, but, practically speaking, you probably want to add some extra constraints. For instance, since your goal is to first test a marketing campaign before rolling it out to the entire market, you probably don't want to choose a lot of treatment cities. A big treatment group would treat almost everyone and kind of defeat the purpose of the test.

Moreover, in many markets, the size of cities tends to follow an exponential distribution, with a small number of cities accounting for almost all of the market (see [Figure 10-1](#)). If you use the canonical synthetic control formulation for this, you would be forced to include the biggest cities, as you are not allowed to extrapolate.

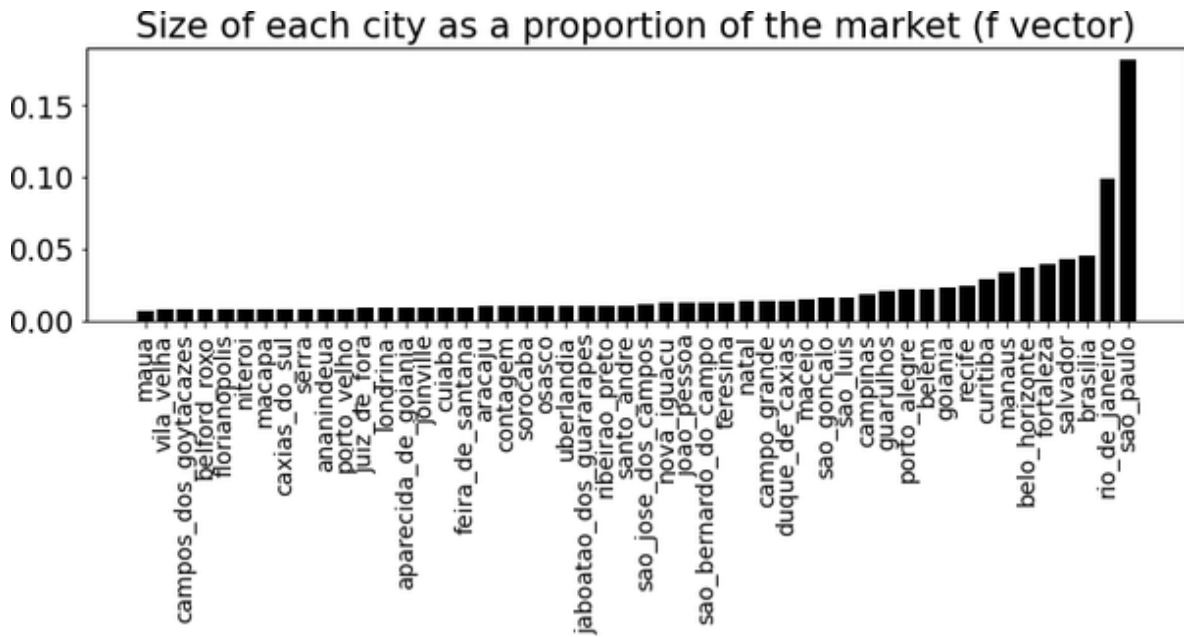


Figure 10-1. There are usually few big cities and many smaller ones

As a consequence, you probably want to allow for an intercept shift in the synthetic control model. Putting these extra requirements together, combining both objectives and adding the synthetic control constraints, you'll end up with the following objective:

$$\begin{aligned}
 \min_{w,v} \quad & ||\mathbf{Y}_{pre}\mathbf{f} - \mathbf{Y}_{pre}\mathbf{w}_{tr} - \alpha_0||^2 + ||\mathbf{Y}_{pre}\mathbf{f} - \mathbf{Y}_{pre}\mathbf{v}_{co} - \beta_0||^2 \\
 \text{s.t} \quad & \sum w_i = 1 \text{ and } \sum v_i = 1, \\
 & w_i, v_i \geq 0 \quad \forall i, \\
 & w_i v_i = 0 \quad \forall i, \\
 & |\mathbf{w}|_0 \leq m
 \end{aligned}$$

I know it seems complicated, but it really isn't. First, it adds intercept terms,  $\alpha_0$  and  $\beta_0$ , to both the synthetic treatment and synthetic control objectives. Then, it adds a bunch of constraints. The first two rows are simply the traditional synthetic control constraints, which states that the weights

must sum up to 1 and be nonnegative. Next, you have the constraint which states that a city cannot be used for both treatment and control. Finally, you have a constraint on the maximum number of cities,  $m$ , you want on the treatment set.

Before you go running to implement this objective in `cvxpy`, I'm sorry to bring it to you, but it is not convex. But don't worry too much with it. Finding the *best* possible sets of treatment and control cities is not strictly necessary. In most cases, it's pretty easy to find sets that are just *good enough*. So, in favor of simplicity, let's see how to code a more modest optimization.

## Trying a Random Set of Treated Units

First, let's try something incredibly simple, which is just selecting a random set of cities. In order to do that, let's first define some constants:  $\mathbf{Y}_{pre}\mathbf{f}$ , the market average you'll try to approximate, a list of all possible cities and  $m$ , the maximum number of treatment cities. I'm setting this last one to 5, but that depends on your business constraints. If your budget only allows for, say, three treatment cities, feel free to change that number:

```
In [5]: y_avg = df_piv.dot(f)
        geos = list(df_piv.columns)
```

```
n_tr = 5
```

Next, let's select a random set of five cities and see how well they perform as a possible treatment set:

```
In [6]: np.random.seed(1)
        rand_geos = np.random.choice(geos, n_tr)
        rand_geos
```

```
Out[6]: array(['manaus', 'recife', 'sao_bernard',
               'salvador', 'aracaju'], dtype='<U')
```

These are the possible treatment cities, but you still need to find the weights for them. If some of those weights turns out to be zero, you won't need to use all five.

To get the weights, I'm using the `SyntheticControl` class that allows for an intercept shift, which was implemented in the previous chapter. The idea is to use the synthetic control to predict `y_avg` by using just these five cities:

```
In [7]: def get_sc(geos, df_sc, y_mean_pre):
```

```
model = SyntheticControl(fit_intercept=True)
model.fit(df_sc[geos], y_mean_pre)

selected_geos = geos[np.abs(model.weights) > 0.05]

return {"geos": selected_geos, "loss": model.loss}

get_sc(rand_geos, df_piv, y_avg)
```

```
Out[7]: {'geos': array(['salvador', 'aracaju'],
                        dtype=object),
         'loss': 1598616.80875266}
```

Once that is done, you can inspect the estimated weights and select the cities with weights that are not too close to zero. I'm wrapping all of that in a function that will allow you to try many different samples and see which one performs better. It is also worth storing the synthetic control loss, as minimizing it will be your objective.

When you fit a synthetic control model using those five cities, you can see that it chooses only two of them as treatment. The other three can be added back to the pool of possible cities and can compose the control group.

With that, you are ready to move to the next step, which is finding both the synthetic treatment and synthetic control:

```
In [8]: def get_sc_st_combination(treatment_geos, df_sc, treatment_result):  
  
        treatment_result = get_sc(treatment_geos, df_sc)  
  
        remaining_geos = df_sc.drop(  
            columns=treatment_result["geos"],  
        ).columns  
  
        control_result = get_sc(remaining_geos, df_sc)  
  
        return {"st_geos": treatment_result["geos"],  
                "sc_geos": control_result["geos"],  
                "loss": treatment_result["loss"]}  
  
resulting_geos = get_sc_st_combination(treatment_geos, df_sc, treatment_result)
```

First, you need to call `get_sc` on the random sample of cities, just like before. This will give you the treatment cities and synthetic treatment loss. Next, you'll figure out what cities are not selected for the synthetic treatment units and call `get_sc` again, passing the remaining cities. This second call to `get_sc` will give you the control

cities and the synthetic control loss. If you sum the two losses, you'll end up with the total loss you wish to minimize.

As expected, calling `get_sc_st_combination` with the same five cities will give you the same treatment units as before:

```
In [9]: resulting_geos.get("st_geos")
```

```
Out[9]: array(['salvador', 'aracaju'], dtype='<
```

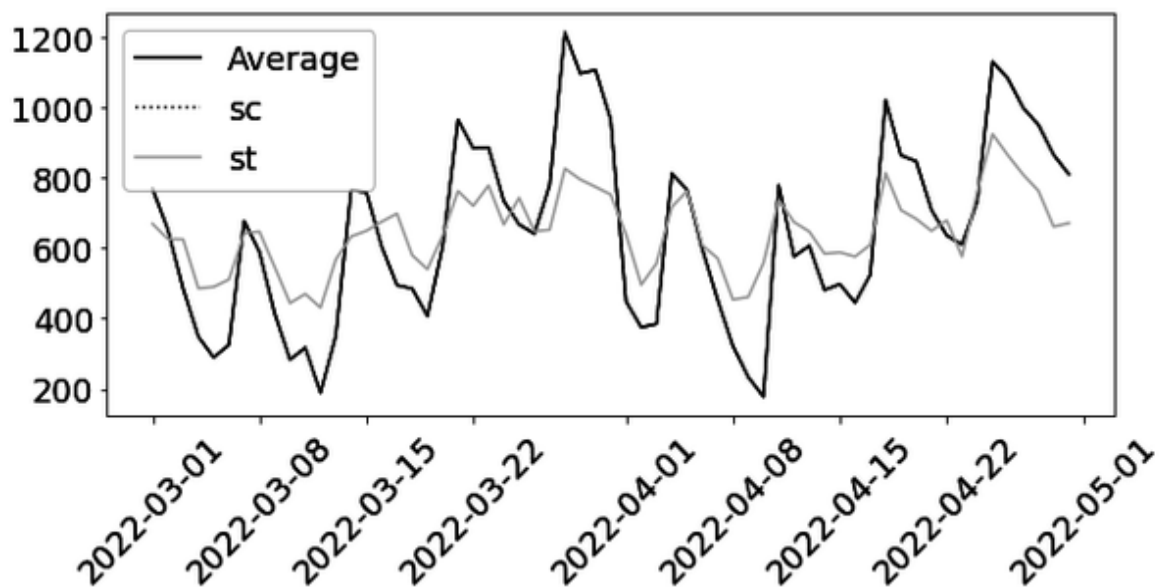
Interestingly, the sum of the treatment and control cities amount to the entire list of 50 cities. But that should not be surprising, especially when  $m$  is small:

```
In [10]: len(resulting_geos.get("st_geos")) + ]
```

```
Out[10]: 50
```

If  $m = 0$  (no treatment units), the obvious solution would be to choose all the cities as a control and set  $\mathbf{v} = \mathbf{f}$ . For  $m > 0$ , but still pretty small, selecting all the cities and slightly

adjusting the weights tends to be the optimal choice. This also means that the majority of the total loss will come from the synthetic treatment objective, since the synthetic control will tend to reconstruct the average market behavior pretty well when  $m$  is small. So much so that when plotting both synthetic treatment and control alongside the market average you can see that, while the synthetic treatment has a poor fit, the synthetic control matches the average almost exactly:



## Random Search

Let's see how to improve upon the result you just got. Since you have a way to calculate the total loss given the number of cities, you can devise many fancy methods to minimize that loss. But since I'm very fond of simplicity, what I suggest is



just randomly searching many combinations and picking one that performs OK. The following code does exactly that. It first generates 1,000 sets of 5 cities and stores everything in the `geo_samples` list. Then, it partially applies `get_sc_st_combination` to the data and average market outcome argument. Finally, it applies that function to the 1,000 sets of cities, all of that in parallel:

```
In [11]: from joblib import Parallel, delayed
         from toolz import partial

         np.random.seed(1)
         geo_samples = [np.random.choice(geos,
                                         for _ in range(1000)]

         est_combination = partial(get_sc_st_co
                                   df_sc=df_piv
                                   y_mean_pre=y

         results = Parallel(n_jobs=4)(delayed(e
                                   for geos
```

It's worth mentioning that this approach is not optimal, but it does tend to produce reasonable sets of treatment cities.

---

## OPTIMIZATION

Formulating the treatment city selection problem like you just did is a simplification on top of the one presented in the paper “Synthetic Controls for Experimental Design,” by Abadie and Zhao. The paper suggests using enumeration or converting the optimization to a constrained quadratic programming problem. Both approaches take a significant amount of time to run, which is why I recommend checking if a simple random search isn’t enough to find good treatment cities.

I myself have also experimented with genetic algorithms for this problem and found that they tend to achieve a better result than simple random search while using the same number of iterations. Additionally, the paper “Designing Experiments with Synthetic Controls,” by Doudchenko et al., proposes a simulated annealing procedure to select the best set of cities. If you find that a simple random search isn’t good enough, I recommend you try those algorithms.

---

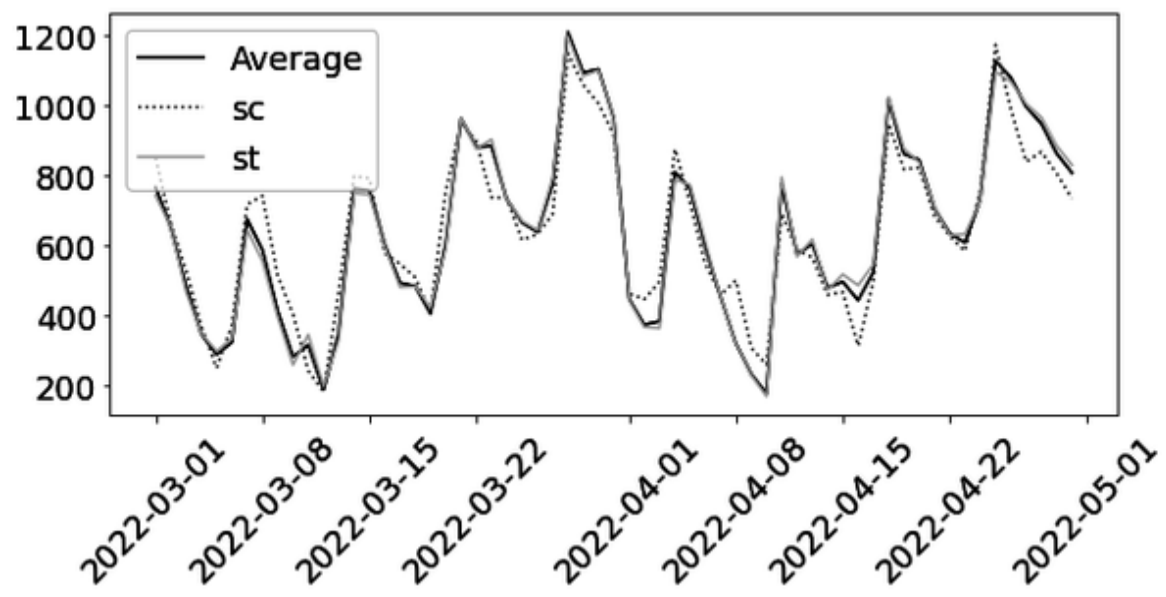
Inspecting the selected treatment cities, you can see that the model chooses only four cities. Not surprisingly, the biggest city, Sao Paulo, is among them. This tends to happen because the biggest cities compose a big chunk of the total market average, so including them in the treated group tends to

reduce the loss a lot. If you wish to avoid this, you can always exclude the largest city from the possible control units:

```
In [12]: resulting_geos = min(results, key=lambda x: x.get("st_geos"))
```

```
Out[12]: array(['sao_paulo', 'florianopolis', 'porto_alegre'],  
              dtype='<U23' )
```

Plotting the synthetic control and treatment once more, you can see how this approach, despite its simplicity, can give you a solution that works pretty well, with both synthetic treatment and control tracking the market average pretty closely:



---

## OTHER EXPERIMENT OBJECTIVES

The idea of minimizing the sum of the synthetic treatment and control objective is so that you can maximize the external validity of the experiment. By selecting a treatment group that closely resembles the market average, the hope is that, with a small experiment, you can learn what would happen if the treatment was implemented nationwide. This is not the only possible objective, though. The paper “Designing Experiments with Synthetic Controls,” by Doudchenko et al., focuses on maximizing the power of the experiment by selecting a set of units with low out-of-sample error. A subsequent paper, “Synthetic Design: An Optimization Approach to Experimental Design with Synthetic Controls,” by Doudchenko et al., casts the treatment units selection into a mixed-integer programming formulation to find the units that minimize the root mean squared error of the effect estimate.

---

Finally, it is worth mentioning that, even though you designed an experiment using a synthetic control design, you don’t necessarily need to read its result with synthetic control, although that is surely a reasonable idea. For instance, you could use synthetic diff-in-diff, as that tends to reduce the variance of the estimator. You do have to be careful on how to estimate the variance of the resulting

estimator, though. Since the group of selected cities is not random, inference procedures based on reassignment of the treatment to different units are not valid. Fortunately, the t-test you learned in the previous chapter doesn't make an assumption on how the units were selected, so you can use it here.

## Switchback Experiment

Synthetic control experiment designs are great for when you have a small number of units and you want to select the best set of them to compose a treatment group. However, to do that, you still need a somewhat reasonable amount of units. But what happens if you only have, say, four units or even one unit? To give an example, suppose you are a small food delivery marketplace that operates within a single city. This company uses dynamic pricing to regulate the supply and demand of the food delivery marketplace, and it wants to know how an increase in delivery fee can impact delivery time by attracting more drivers to the fleet while throttling or postponing customer demand. Notice that traditional A/B testing wouldn't work here. Increasing the price for 50% of the customers would also benefit those in the control group, since the overall demand would fall, increasing the number of available drivers. Also, synthetic control experiments wouldn't work either, because the company operates in just

one city. But there is one type of experiment design that might just do in this situation.

If the effect of rising prices dissipates rather quickly once they go back to the normal level, the company can turn the price increases on and off multiple times and do a sequence of before-and-after comparisons. This approach is named *switchback experiments*, and it is great for when you have just one or a very small number of units. But for it to work, *the order of the carryover effect must be small*. That is, the effect of the treatment cannot propagate to many periods after the treatment. For instance, in the food delivery case, increasing prices tend to cause an increase in supply shortly after; when prices are back to normal, the excess supply dissipates in a few hours. Hence, the order of the carryover effect is small, so switchback experiments are an interesting proposition.

Before talking about designing a switchback experiment, since this is the first time they appear in this book, I think it's worth walking through one for you to grasp how they work. The following data frame contains data from a switchback experiment with 120 periods, where each period is 1 hour. In this experiment, *the treatment was randomized at every time period, with a 50% chance of selecting the treatment or control*. The `d` column tells you if the price increase (treatment) was on or off at that hour and the `delivery_time` is the outcome of interest.

Additionally, I've added three columns that would not be observed in reality, but should help your understanding of what is going on. `delivery_time_1` is the delivery time you would get if the treatment was always on and `delivery_time_0`, if it was always off. The difference between them, `tau`, is the total effect of the treatment and it is usually the causal quantity of interest in a switchback experiment. Since the treatment decreases delivery time, the effect is negative. Also, due to carryover, the effects on the first two periods are smaller:

```
In [13]: df = pd.read_csv("../data/sb_exp_every.")
         df.head()
```



|   | d | delivery_time | delivery |
|---|---|---------------|----------|
| 0 | 1 | 2.84          | 2.84     |
| 1 | 0 | 4.49          | 1.49     |
| 2 | 0 | 7.27          | 2.27     |
| 3 | 1 | 5.27          | 2.27     |
| 4 | 1 | 5.59          | 4.59     |

Figure 10-2 shows that the observed delivery time fluctuates between the delivery time you would have with the treatment always on and always off. Moreover, after three consecutive equal treatments, the observed outcome matches the one you would have with the treatment always on or always off.

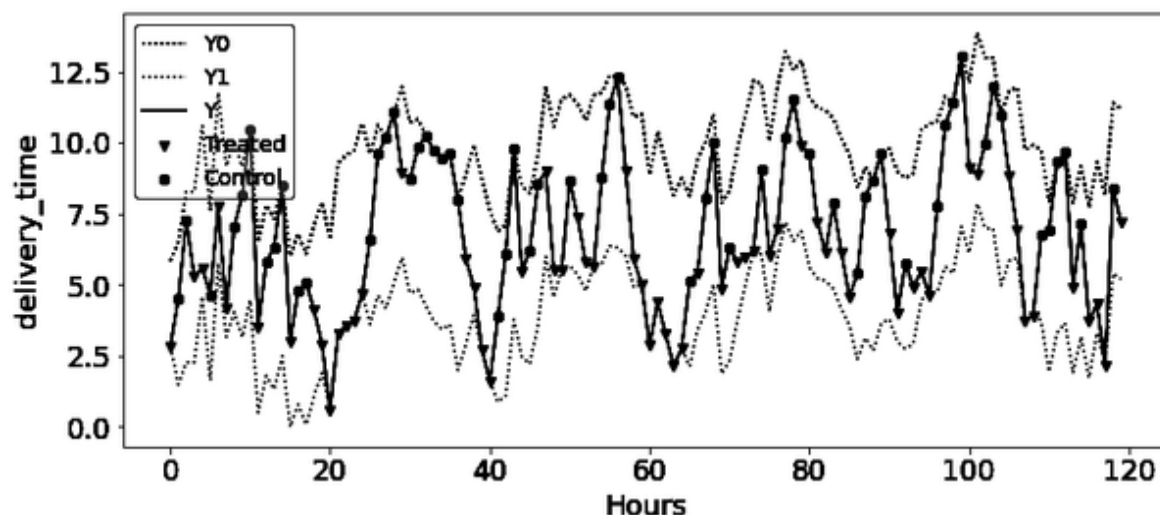


Figure 10-2. In a switchback experiment, the observed outcome fluctuates between the always treat and never treat potential outcomes

Take  $T = 20$  to  $T = 23$ , for example. At those points, due to chance, the treatment was on for three or more consecutive periods and the delivery time matches the delivery time you would have under the treatment being always on.

Conversely, at around  $T = 32$ , you can see a sequence where the treatment has been off (control) for three or more consecutive periods. At that point, the outcome matches the outcome you would have if the treatment was always off. If the treatment was on or off for less than three consecutive periods, the observed outcome is somewhere in the middle. This tells you that, in this case, the outcome depends on the treatment of three periods: the immediate treatment and the treatment from two periods before:

$$Y_t = f(d_{t-2}, d_{t-1}, d_t)$$

In other words, the order of the carryover effect is 2.

Of course, in reality, you wouldn't know this, since you can only see the observed outcome. But don't worry about that too much. I'll show you how to estimate the size of the carryover period. For now, I just want you to get an intuitive understanding of switchback experiments. On top of that, you'll be able to develop a more formal language to describe what is going on.

## Potential Outcomes of Sequences

Since the effect of the treatment carries over to subsequent time (2 in this case), when it comes to switchback experiments, the potential outcome has to be defined in terms of a vector of treatments,

$Y_t(\mathbf{D}) = Y_t([d_0, d_1, d_2, \dots, d_T])$ . Fortunately, you can simplify this with two assumptions. First, if you assume no anticipation of the treatment, the potential outcome will only depend on current and past treatment, but not on future ones. As a result, you can write

$Y_t(\mathbf{D}) = Y_t([d_0, d_1, d_2, \dots, d_t])$ . If you know the size of the carryover period  $m$ , you could write it as

$Y_t(\mathbf{D}) = Y_t([d_{t-m}, \dots, d_t])$ . Since  $m = 2$  here, the potential outcome simplifies to  $Y_t(\mathbf{D}) = Y_t([d_{t-2}, d_{t-1}, d_t])$ , which is a lot simpler than what you would have with no assumptions at all.

Having defined those potential outcomes, you can write the total effect of the treatment as:

$$\tau_m = E[Y_t(1_{t-m}, \dots, 1_t) - Y_t(0_{t-m}, \dots, 0_t)]$$

This is the effect of going from an always off treatment to an always on treatment. In the case where  $m = 2$ , this would be  $E[Y_t(1, 1, 1) - Y_t(0, 0, 0)]$ . Since all of that simplification requires you to know  $m$ , let's turn your attention to that now.

## Estimating the Order of Carryover Effect

Let's say that you have some business expert knowledge that puts an upper bound on  $m$ . For instance, you know that the effect of price increases doesn't last for more than 6 hours. In that case, you can regression-estimate the model:

$$y_t = \alpha + d_t + d_{t-1} + \dots + d_{t-K} + e_t$$

and read the parameter estimates' size and significance. The order of the carryover effect will depend on which parameters are statistically significant and also have a large impact on the outcome.

Notice that this imposes another assumption, which is that the effects of the lags are additive:

$$Y_t = f(d_t, d_{t-1}, d_{t-2}) = \alpha + d_t + d_{t-1} + d_{t-2} + e_t$$

To create lags of the treatment, you can use the

`.shift(lag)` method from pandas.

To programmatically create six lags, I'm taking advantage of the fact that the `.assign(...)` method takes as

argument the name of the new column you want to create and that, in Python, you can pass named argument with

```
** and a dictionary. So, for instance,  
df.assign(a=1, b=1) is the same as  
df.assign(**{"a":1, "b":2}) :
```

```
In [14]: df_lags = df.assign(**{  
        f"d_l{l}" : df["d"].shift(1) for l in range(3)  
        })  
  
df_lags[[f"d_l{l}" for l in range(7)]]
```

|   | d_l0 | d_l1 | d_l2 |
|---|------|------|------|
| 0 | 1    | NaN  | NaN  |
| 1 | 0    | 1.0  | NaN  |
| 2 | 0    | 0.0  | 1.0  |
| 3 | 1    | 0.0  | 0.0  |
| 4 | 1    | 1.0  | 0.0  |

Once you have the data with its lags, all you have to do is regress the outcome on the lags and the current treatment (which can be thought of as lag 0). Notice that statsmodels will drop the rows with NaNs:

```
In [15]: import statsmodels.formula.api as smf

        model = smf.ols("delivery_time ~" + "+".join("d_l"+str(i)
                                                       for i in range(1, 17)),
                        data=df_lags).fit()

        model.summary().tables[1]
```

|                  | coef    | std err | t      | P> t  |
|------------------|---------|---------|--------|-------|
| <b>Intercept</b> | 9.3270  | 0.461   | 20.246 | 0.000 |
| <b>d_l0</b>      | -2.9645 | 0.335   | -8.843 | 0.000 |
| <b>d_l1</b>      | -1.8861 | 0.339   | -5.560 | 0.000 |
| <b>d_l2</b>      | -1.0013 | 0.340   | -2.943 | 0.004 |
| <b>d_l3</b>      | 0.2594  | 0.341   | 0.762  | 0.448 |
| <b>d_l4</b>      | 0.1431  | 0.340   | 0.421  | 0.675 |
| <b>d_l5</b>      | 0.1388  | 0.340   | 0.408  | 0.684 |
| <b>d_l6</b>      | 0.5588  | 0.336   | 1.662  | 0.099 |

By looking at the lag parameters, you can see that they are significant up to lag 2, which indicates a carryover effect of 2.

Interestingly, with the regression model, you don't need to know the correct order of the carryover effect  $m$  to estimate the total effect  $\tau_m$ . As long as your regression contains more lags than the correct  $m$ , you can just sum up all the lag parameter estimates:

$$\hat{\tau}_m = \sum_{l=0}^{lags} \hat{d}_{t-l}.$$

To get the variance, you also have to sum up the variance of each individual lag:

```
In [16]: ## remember to remove the intercept
tau_m_hat = model.params[1:].sum()
se_tau_m_hat = np.sqrt((model.bse[1:].sum()))
print("tau_m:", tau_m_hat)
print("95% CI:", [tau_m_hat - 1.96*se_tau_m_hat,
                  tau_m_hat + 1.96*se_tau_m_hat])
```

```
Out[16]: tau_m: -4.751686115272022
          95% CI: [-6.5087183781545574, -2.9946558523894865]
```

Since you are using a bunch of lags, the total effect estimate will be rather imprecise. But if you settle for two lags, you can reduce the variance substantially:

```
In [17]: ## selecting lags 0, 1 and 2
        tau_m_hat = model.params[1:4].sum()
        se_tau_m_hat = np.sqrt((model.bse[1:4]
        print("tau_m:", tau_m_hat)
        print("95% CI:", [tau_m_hat -1.96*se_tau_m_hat,
                           tau_m_hat +1.96*se_tau_m_hat])
```

```
Out[17]: tau_m: -5.8518568954422925
        95% CI: [-7.000105171362163, -4.703608819522421]
```

## Design-Based Estimation

The previous procedure depends on a correct specification of the model for the potential outcome  $Y_t(\mathbf{D})$ . Since it's a time series, this is not a trivial task. One alternative is to estimate  $\tau_m$  with something like inverse propensity weighting (IPW), which would only require knowledge of how the treatment was assigned. Since that is controlled by the company designing the experiment, this second approach relies on more plausible assumptions. It will only require you to know the carryover effect order,  $m$ .

Remember that IPW reconstructs a potential outcome by scaling up the observed outcome by the inverse treatment



probability  $E[\hat{Y}_d] = N^{-1} \sum (Y_d \mathbb{1}(D = d) / P(D = d))$ .

You'll do the same thing, but now you have to take into account that the potential outcome is defined in terms of a vector of treatment. For instance, in the case that the effect carries over for two periods,  $m = 2$ , you want to reconstruct  $Y_t(0, 0, 0)$  and  $Y_t(1, 1, 1)$ , which requires you to calculate the running probability of observing three equal treatments in a row. For  $m = 2$ , that would be

$P(d_{t-2} = d, d_{t-1} = d, d_t = d)$ , or, more generally:

$$P(\mathbf{D}_{t-m:t} = \mathbf{d}),$$

where  $\mathbf{D}_{t-m:t}$  is the vector of the current and last  $m$  treatments and  $\mathbf{d}$  is a vector of constant treatment  $d$ . Here, you can focus on the case where the randomization probability  $p$  is always 50%. That is, at each randomization point, the treatment has a 50% chance of staying the same or switching. In terms of experiment design, this will increase the power of the experiment, as it increases the treatment variance. From an estimation procedure, it makes it so that  $P(\mathbf{D}_{t-m:t} = \mathbf{1}) = P(\mathbf{D}_{t-m:t} = \mathbf{0})$ . However, this does not mean that  $P(\mathbf{D}_{t-m:t} = \mathbf{d})$  will be the same everywhere. In fact that probability depends on the randomization points and where you are on the sequence. In your food delivery example,  $p = 50\%$  and the treatment was randomized at every time period. So, the running probability of observing three consecutive treatments, like in the sequence

---

`[1, 1, 1, 1, 1, 1]` , is  
`[na, na, .5^3, .5^3, .5^3, .5^3]` . However, if you randomize every three periods, the same sequence has a running probability of `[na, na, .5, .5^2, .5^2, .5]` . That's because, at  $t = 4, 5$  the lag 2 window, which contains the current time period and the previous two, will contain two randomization points.

I hope that is easy to conceptualize, but, unfortunately, it is not as easy to code. I'll do my best to explain how it's done, but it does require some clever array manipulation. But maybe this will teach you some new NumPy tricks. To make things more tangible, let's try to compute the running probability of observing  $m + 1$  equal consecutive treatments in the case where the randomization happens every three periods:

```
In [18]: rad_points_3 = np.array([True, False,
                                   rad_points_3
```

```
Out[18]: array([ True, False, False,  True, Fal
```

The first step is identifying the randomization windows from the randomization points. You can take advantage of the fact

that `True` is interpreted as 1 and `False`, as 0. If you compute the cumulative sum of the randomization points, the sum will increase by 1 at each randomization point:

```
In [19]: rad_points_3.cumsum()
```

```
Out[19]: array([1, 1, 1, 2, 2, 2])
```

Now, you can view each randomization window as the sequence of equal integer numbers. You have two randomization windows of size 3 each.

The next step is to compute the carryover window, which will be of size  $m + 1$ . Notice that, in this case, the randomization window is equal to the carryover window, but this is not generally the case, so the code has to work for different  $ms$ . One way to do that is to use the function

`sliding_window_view`, from NumPy, which, as the name suggests, creates a running window array. Notice that this function discards the first  $m$  windows, as they would not be complete:

```
In [20]: from numpy.lib.stride_tricks import sl
```

```
m = 2
sliding_window_view(rad_points_3.cumsu
```

```
Out[20]: array([[1, 1, 1],
                [1, 1, 2],
                [1, 2, 2],
                [2, 2, 2]])
```

From these windows, you can calculate how many randomization windows are contained in each carryover window. It is simply the quantity of different numbers in each carryover window. Unfortunately, there isn't a NumPy function that counts the unique elements across an axis, so you'll have to build one on your own. To do that, you can use the function `np.diff`, which counts the difference of subsequent entries in an array:

```
In [21]: np.diff(sliding_window_view(rad_points
```

```
Out[21]: array([[0, 0],
                [0, 1],
                [1, 0],
```

```
[0, 0]])
```

Then, finally, summing up the columns and adding 1 returns the number of randomization windows at each point of the original array. Notice that the result starts at index 2 ( $T = 3$ ), since `sliding_window_view` discards the first  $m$  windows. In this example, at  $T = 3$ , the last three entries contain a single randomization window, at time  $T = 4$ , it contains two randomization windows, and so on. To avoid any confusion, you can prepend  $m$  `np.nan` at the beginning of the array:

```
In [22]: n_rand_windows = np.concatenate([
        [np.nan]*m,
        np.diff(sliding_window_view(rad_pc
        axis=1).sum(axis=1)+1
    ])

    n_rand_windows
```

```
Out[22]: array([nan, nan,  1.,  2.,  2.,  1.])
```

Now, to get the probability vector, all you have to do is take the probability of the experiment, in this case, 0.5, and exponentiate it by the preceding array:

```
In [23]: p=0.5  
         p**n_rand_windows
```

```
Out[23]: array([ nan,  nan,  0.5,  0.25,  0.25,  0.5])
```

Here is everything wrapped up in a function. You can also check that this logic works for other randomization frequencies, like randomizing every period:

```
In [24]: def compute_p(rand_points, m, p=0.5):  
         n_windows_last_m = np.concatenate(  
             [np.nan]*m,  
             np.diff(sliding_window_view(rand_points,  
                 axis=1).sum(axis=1)+1,  
                 m))  
         return p**n_windows_last_m  
  
         compute_p(np.ones(6)==1, 2, 0.5)
```

```
Out[24]: array([ nan, nan, 0.125, 0.125, 0.125
```

and even for nonregular randomization intervals:

```
In [25]: rand_points = np.array([True, False, True, True])
         compute_p(rand_points, 2, 0.5)
```

```
Out[25]: array([ nan, nan, 0.5, 0.25, 0.25, 0.25
```

But all of that was just to compute  $P(\mathbf{D}_{t-m:t} = \mathbf{d})$ . You still have to take a look at the rest of the estimator for the potential outcome:

$$\hat{Y}(\mathbf{d}) = \frac{1}{T-m} \sum_{t=m+1}^T Y_t \frac{\mathbb{1}(\mathbf{D}_{t-m:t} = \mathbf{d})}{P(\mathbf{D}_{t-m:t} = \mathbf{d})}$$

In other words, this estimator will scale up the observed outcome by the running probability you just learned how to compute, whenever the last  $m$  treatment and the current one are all the same. Then, it will average those scaled-up outcomes.

To code this, the only missing piece is the indicator function in the numerator, which evaluates to `true` whenever the last  $m + 1$  treatments are equal to  $d$ . Fortunately, now that you know about `sliding_window_view`, that is pretty easy to do. First, create the  $m + 1$  window view of the treatment array. Then, check if all the elements in that window are equal to the treatment. Here is a function to do just that:

```
In [26]: def last_m_d_equal(d_vec, d, m):  
         return np.concatenate([  
             [np.nan]*m,  
             (sliding_window_view(d_vec, m+1) == d).all()  
         ])  
  
         print(last_m_d_equal([1, 1, 1, 0, 0, 0], 1, 2))  
         print(last_m_d_equal([1, 1, 1, 0, 0, 0], 0, 2))
```

```
Out[26]: [nan nan  1.  0.  0.  0.]  
         [nan nan  0.  0.  0.  1.]
```

Applying this function to the treatment vector

`[1, 1, 1, 0, 0, 0]`, using  $m = 2$  and trying to find when the current and last two entries are treated ( $d = 1$ ) returns a 1 only at the third entry, as it should be.



You are finally ready to join all those pieces into the IPW estimator for switchback experiments:

$$\hat{\tau} = \frac{1}{T-m} \sum_{t=m+1}^T \left\{ Y_t \left( \frac{\mathbb{I}(\mathbf{D}_{t-m:t} = \mathbf{1})}{P(\mathbf{D}_{t-m:t} = \mathbf{1})} - \frac{\mathbb{I}(\mathbf{D}_{t-m:t} = \mathbf{0})}{P(\mathbf{D}_{t-m:t} = \mathbf{0})} \right) \right\}$$

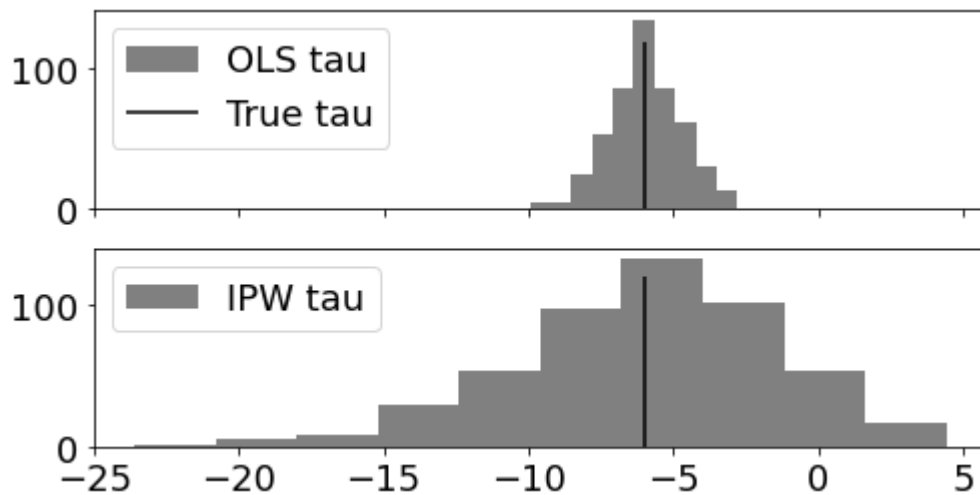
```
In [27]: def ipw_switchback(d, y, rand_points,
                               p_last_m_equal_1 = compute_p(rand_points, 1),
                               p_last_m_equal_0 = compute_p(rand_points, 0),
                               last_m_is_1 = last_m_d_equal(d, 1, m),
                               last_m_is_0 = last_m_d_equal(d, 0, m),
                               y1_rec = y*last_m_is_1/p_last_m_equal_1,
                               y0_rec = y*last_m_is_0/p_last_m_equal_0):
    return np.mean((y1_rec-y0_rec)[m:])
```

Let's now use this function to estimate  $\tau$  from the price increase switchback experiment. Remember that the treatment was randomized at each time period, so you can pass a vector of `True`s, `np.ones(len(df))==1`, to the `rand_points` arguments:

```
In [28]: ipw_switchback(df["d"],  
                        df["delivery_time"],  
                        np.ones(len(df)) == 1,  
                        m=2, p=0.5)
```

```
Out[28]: -7.426440677966101
```

This estimated effect is a bit lower (meaning that prices decrease waiting time more) than the one you got with OLS. Also, it is worth mentioning that it has a much higher variance. In the following plot, I've simulated 500 switchback experiments just like the one you have and computed both the OLS and IPW estimates for the total effect. As you can see, both methods are unbiased, since the average of the estimated  $\hat{\tau}$  matches  $\tau$  in both cases. However, the IPW distribution is much more spread out:



## Optimal Switchback Design

I think we can all agree that more variance is undesirable from an estimation standpoint. However, if you plan to design a switchback experiment, you probably want to be conservative and look at the worst possible case. Also, you likely want to make the least amount of assumptions possible. For this reason, taking the IPW estimator and trying to come up with an experiment design that will reduce its variance is an attractive proposition. First, let's think on an intuitive level how you might go about doing that.

From the IPW estimator formula, you know that it only keeps the sequences with  $m + 1$  consecutive equal treatments. This means it throws away any  $m + 1$  sequence that has more than one treatment assignment. If you go back to the plot that shows the observed and potential outcomes of your experiment, it would mean throwing away all the data in

between the upper and lower potential outcomes. Hence, if you want to use more data, all you have to do is make sure you have more consecutive equal treatments. At the limit, you would just set all the treatment sequences to either 0 or 1. This would maximize the usable data; however, it would decrease the variance of the treatment—and, in this extreme case, make estimation impossible. Since the variance of the estimator can be decreased by having both more useful data and higher treatment variance, you have to find a balance between the two.

Intuitively speaking, one way to do that is by randomizing every  $m + 1$  period. So, in your example, if the order of the carryover effect is 2, you would randomize every three periods. This is indeed very close to the design that minimized variance, but it is not exactly it. Turns out you can improve it slightly if you instead randomize at every  $m$  periods and add a gap of size  $m$  at the beginning and end of the experiment horizon, when  $m > 0$ :

$$\mathbb{T}^* = \{1, 2m + 1, 3m + 1, \dots, (n - 2)m + 1\},$$

where  $\mathbb{T}^*$  are the optimal randomization points,  $m$  is the order of the carryover effect, and  $n$  is an integer  $\geq 4$  such that  $T/m = n$ . In practice, this means that the length of the experiment has to be divisible by the carryover order and long enough to contain at least four blocks of size  $m$ .

---

**TIP**

When there is no carryover, ( $m = 0$ ), the optimal design is just randomizing at every period, which would maximize treatment variance while keeping all the data.

---

To consolidate your understanding about this, let's look at some examples. First, if  $T = 12$  and  $m = 2$ , you would randomize at  $t = 1$ , leave a gap of size 2 at  $t = 3$  then randomize again at  $t = 5, 7, 9$ , and leave a final gap of size 2 at  $t = 11$ :

```
In [29]: m = 2
         T = 12
         n = T/m
         np.isin(
             np.arange(1, T+1),
             [1] + [i*m+1 for i in range(2, int(
                 ) * 1
```

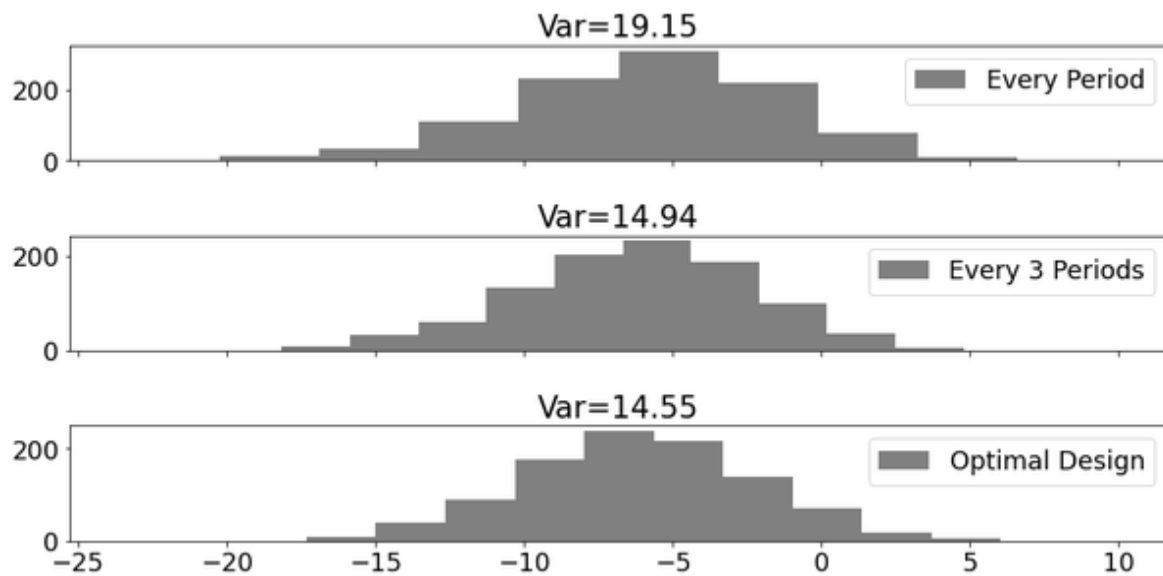
```
Out[29]: array([1, 0, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0])
```

When  $m = 3$  and  $T = 15$ , you again randomize at  $t = 1$ , then leave a gap of size 3 at  $t = 4$ , randomize again at  $t = 7, 10$ , and leave a final gap of size 3 at  $t = 13$ :

```
In [30]: m = 3
         T = 15
         n = T/m
         np.isin(
             np.arange(1, T+1),
             [1] + [i*m+1 for i in range(2, int(T/m))]
         ) * 1
```

```
Out[30]: array([1, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0])
```

Now, as interesting as this is, it is worth mentioning that the variance decrease you'll probably get with this optimal design is nowhere as near as the one you would get by making a model assumption and using OLS. In the following plots, I've simulated 500 experiments with  $T = 120$  and  $m = 2$  using the optimal design, the intuitive design (randomizing every three periods), and randomizing every period. Then, I used the IPW estimator to estimate the effect in all of them. As you can see, there is a variance reduction, but it's nothing dramatic:



Still, since the optimal design is rather simple to implement, I don't see any reason not to.

## Robust Variance

We talked a lot about reducing variance, but I haven't yet told you how to estimate the variance of the IPW estimator. That's because it actually depends on the design of your experiment. Now that we've set on an optimal design, you can compute the variance that it entails. So, for the sake of the example, let's say that the food delivery company ran a second experiment, a lot like the first one, with  $T = 120$ ,  $p = 0.5$ , but now they decided the randomization points based on the optimal design. The data from that experiment is stored in the following data frame:

```
In [31]: df_opt = pd.read_csv("./data/sb_exp_or")
df_opt.head(6)
```

|   | rand_points | d | delivery_time |
|---|-------------|---|---------------|
| 0 | True        | 0 | 5.84          |
| 1 | False       | 0 | 5.40          |
| 2 | False       | 0 | 8.86          |
| 3 | False       | 0 | 8.79          |
| 4 | True        | 0 | 10.93         |
| 5 | False       | 0 | 7.02          |

The IPW function from before is pretty general, so you can use it here to estimate the treatment effect:

```
In [32]: tau_hat = ipw_switchback(df_opt["d"],
                                     df_opt["deliv"],
                                     df_opt["rand_"])
```



```
m=2, p=0.5)
```

```
tau_hat
```

```
Out[32]: -9.921016949152545
```

Now, for the variance. The formula is a bit ugly, but you'll see it's not that complicated, once I parse it for you. First, let's partition your data into  $K = T/m$  blocks such that each block has size  $m$ . Then, define the sum of the outcomes of a block  $k$  as  $\bar{Y}_k = \sum Y_{km+1:(k+1)m}$ . For instance, if  $Y = [1, 1, 1, 2, 2, 3]$ , for  $m = 2, k = \{1, 2\}$ ,  $\bar{Y} = [\sum Y_{3:4}, \sum Y_{5:6}] = [3, 5]$ . Notice that the first block is thrown away. Having defined the sum of those blocks, a conservative estimate to the variance is as follows:

$$\hat{\sigma}(\hat{\tau}) = \frac{1}{(T - m)^2} \left\{ 8\bar{Y}_1^2 + \sum_{k=2}^{K-1} 32\bar{Y}_k^2 \mathbb{1}(d_{km+1} = d_{(k+1)m+1}) + 8\bar{Y}_K^2 \right\}$$

Now, let me parse it for you. First, the denominator is the square of the sample size. But since you discarded the first  $m$  entries, you have to subtract  $m$ . The numerator is composed of three main terms. The first and the last term take into account the gap you left at the beginning and end of the optimal design. The term in the middle is a bit more

complicated, because it has this indicator function. That function evaluates to 1 whenever two consecutive blocks have the same treatment. Notice that, because of the gap at the beginning and end, this is sure to happen for the first and last term, which is why you don't need the indicator function there.

To code this formula, you'll first need to make sure that  $T$  is divisible by  $m$  by a factor greater or equal to 4. Then, you'll make use of the functions `hsplit` and `vstack`. The first will partition the array into blocks and the second will pile up the blocks vertically. Here is an example:

```
In [33]: np.vstack(np.hsplit(np.array([1, 1, 1, 2,
```

```
Out[33]: array([[1, 1],
               [1, 2],
               [2, 3]])
```

You can then sum the columns of that piled-up array to get  $\bar{Y}$ .

For the indicator function, you'll do the same thing, but with the treatment vector. Due to the nature of the experiment design, the entire block of treatment will be either 1 or 0, so

you can take just the first column to know which treatment was assigned to that block. To know if two consecutive blocks have the same treatment, use the `diff` function. This will discard yet another block. Here is an example:

```
In [34]: np.diff(np.vstack(np.hsplit(np.array(
```

```
Out[34]: array([False,  True])
```

Now, for the entire variance function:

```
In [35]: def var_opt_design(d_opt, y_opt, T, m)

    assert ((T//m == T/m)
            & (T//m >= 4)), "T must be

    # discard 1st block
    y_m_blocks = np.vstack(np.hsplit(y

    # take 1st column
    d_m_blocks = np.vstack(np.split(d_

    return (
        8*y_m_blocks[0]**2
        + (32*y_m_blocks[1:-1]**2*(np.
```

```

        + 8*y_m_blocks[-1]**2
    ) / (T-m)**2

```

Finally, with that function, you can estimate the variance and place a confidence interval around your effect estimate:

```

In [36]: se_hat = np.sqrt(var_opt_design(df_opt,
                                          df_opt,
                                          T=120,
                                          tau_hat=tau_hat))

[tau_hat - 1.96*se_hat, tau_hat + 1.96*se_hat]

```

```

Out[36]: [-18.490627362048095, -1.3514065362569]

```

That is a pretty wide confidence interval—a lot wider than the earlier one, with OLS and the design that randomized every period. Still, sometimes this extra variance is a price worth paying, if you don't want to make further model assumptions. Moreover, even if you follow the optimized design, you can still analyze it with OLS. Even though this optimal design is not meant to minimize the OLS variance, I

find that it still gives more precise estimates than randomizing at every period, for instance.

---

## FINDING $m$ WITH FEWER ASSUMPTIONS

The optimal switchback experiment design, the IPW estimator and the variance estimator presented in this chapter was taken from the paper “Design and Analysis of Switchback Experiments,” by Bojinov, Simchi-Levi, and Zhao (yes, it’s the same Zhao from the synthetic control design paper). That paper contains a more generic formula for the optimal design, which works when  $T$  is not divisible by  $m$ . Since that formula is a lot more complex and a company can easily make it so that  $T$  is divisible by  $m$ , I chose to omit it here.

Moreover, the paper proposes another procedure to find the order of the carryover effect  $m$ . The idea is based on running two optimal experiments,  $e_1$  and  $e_2$ , each one with a candidate value for  $m$ ,  $m_1$ , and  $m_2$ . Say that  $m_1 < m_2$ . If the effect estimates from both experiments are the same, you cannot reject the hypothesis  $H_0 : m \leq m_1$ . That’s because, if  $m > m_1$ ,  $e_1$  would return a more biased estimate for the effect than  $e_2$ . Hence, you can search for  $m$  based on rejecting the hypothesis that the effect estimates from two experiments are the same,  $H_0 : \hat{\tau}_1 = \hat{\tau}_2$ .

Honestly, I’m not sure I like that procedure. The high variance will make it very hard to reject that null hypothesis, unless the experiment is incredibly long (very high  $T$ ). Still,

that is an alternative to the OLS method I showed you, in case you don't want to make any model-based assumptions.

---

## Key Ideas

This chapter looks at two alternative experiment designs for when the number of experimental units at your disposal is rather short. This can happen, for example, when you are forced to zoom out from treating customers to treating entire cities, as it is often the case in both online and offline marketing.

First, you learned about synthetic control design. Here, the goal is to find a small set of units which, when combined together, approximate the average behavior of all units. This can be done by maximizing the following objective:

$$\begin{aligned} \min_{w,v} \quad & ||\mathbf{Y}_{pre}\mathbf{f} - \mathbf{Y}_{pre}\mathbf{w}_{tr} - \alpha_0||^2 + ||\mathbf{Y}_{pre}\mathbf{f} - \mathbf{Y}_{pre}\mathbf{v}_{co} - \beta_0||^2 \\ \text{s.t} \quad & \sum w_i = 1 \text{ and } \sum v_i = 1, \\ & w_i, v_i \geq 0 \ \forall \ i, \\ & w_i v_i = 0 \ \forall \ i, \\ & |\mathbf{w}|_0 \leq m \end{aligned}$$

where  $\mathbf{f}$  corresponds to the weight each unit contributes to the global average,  $\mathbf{w}$  and  $\mathbf{v}$  are the weights of the synthetic treatment and control units, and  $m$  is a constraint on the maximum number of treatment units.

Synthetic control designs are great for when you have a relatively small number of experimental units, as it allows you to treat just the ones that are good at reproducing the average. Moreover, synthetic control design is well suited for when the order of the carryover effect is large, meaning that the treatment effect takes a long time to dissipate.

If that is not the case—that is, the order of carryover effect is small—then switchback experiments tend to offer a good alternative. Even if you have very few or only one experimentation unit, switchback experiments work by turning the treatment on and off for the same unit and then doing a bunch of before-and-after comparisons.

A switchback experiment is defined by the probability of treatment—which, if you want to maximize power, should be set to 50%—and the randomization points, or time periods when randomization happens. If you know the order of the carryover effect,  $m$ , and it is greater than 0, then the optimal design randomizes every  $m + 1$  time periods:

$$\mathbb{T}^* = \{1, 2m + 1, 3m + 1, \dots, (n - 2)m + 1\},$$

where  $n = T/m$ . For this to work, the length of the experiment  $T$  has to be divisible by  $m$  by more than 4. Also, if there is no carryover effect ( $m = 0$ ), the optimal design is one that simply randomizes at each time period.